deckhouse СТАЧКА

3–4 октября ▪ Санкт-Петербург

Git на котиках

Как работают системы контроля версий



Дизайн и контент

Техническая документация

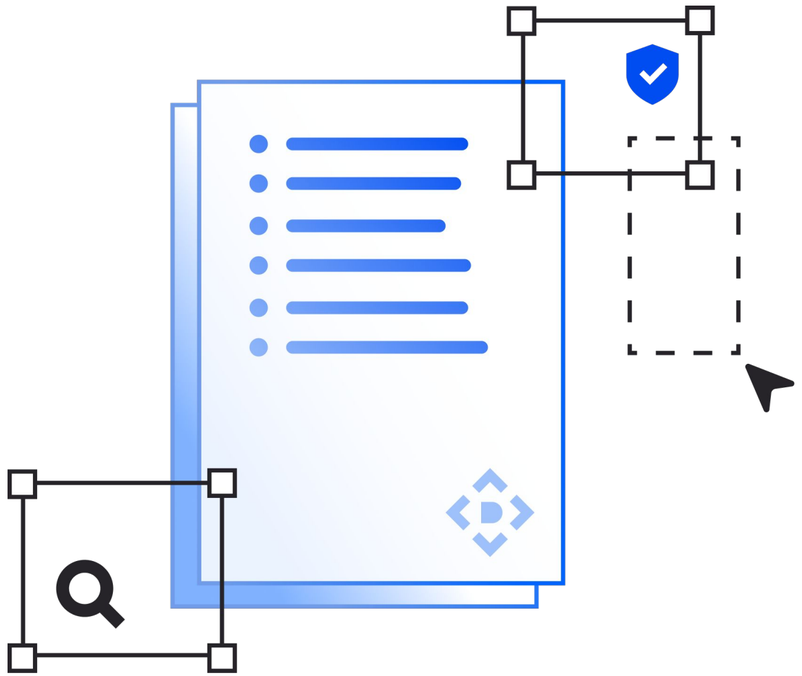
deckhouse СТАЧКА

Константин Нежберт

Технический писатель в команде документации Deckhouse Platform



План доклада



- Что такое Git и системы контроля версий
- С какими файлами **хорошо** и **плохо** работает Git
- Страхи новичков
- Базовые понятия
- Чем работать с Git
- Git и GitHub
- Минимальный рабочий цикл
- Выводы

Как всё начинается?

01 Надо написать документацию к новой фиче.



Как всё начинается?

01 Надо написать документацию к новой фиче.

02 Она в бранче add-new-feature, ещё не смержена.



Как всё начинается?

01 Надо написать документацию к новой фиче.

02 Она в бранче add-new-feature, ещё не смержена.

03 Начни там новую ветку и оформи потом PR в неё.



Как всё начинается?

01 Надо написать документацию к новой фиче.

02 Она в бранче add-new-feature, ещё не смержена.

03 Начни там новую ветку и оформи потом PR в неё.

04 Не забудь перед отправкой ребейзнуть,
вдруг там новое что-то принесли.



Как всё начинается?

01 Надо написать документацию к новой фиче.

02 Она в бранче add-new-feature, ещё не смержена.

03 Начни там новую ветку и оформи потом PR в неё.

04 Не забудь перед отправкой ребейзнуть,
вдруг там новое что-то принесли.



Есть вопросы? Го!

Как всё начинается?

01 Это очень сложно, я ничего не понимаю!



Как всё начинается?

01 Это очень сложно, я ничего не понимаю!

02 Вдруг я всё сломаю?



Как всё начинается?

01 Это очень сложно, я ничего не понимаю!

02 Вдруг я всё сломаю?

03 Нужно помнить 100 команд в терминале...



Как всё начинается?

01 Это очень сложно, я ничего не понимаю!

02 Вдруг я всё сломаю?

03 Нужно помнить 100 команд в терминале...

04 Я всё сломал. Что делать?



Что такое Git?

Git — это система контроля версий

Что такое **система контроля версий**?



- Что такое **система контроля версий**?
- Что такое... **версия**?



Что такое… версия?



Что такое... версия?



Документы



Что такое… версия?



Документы



Игры

Сейв — можно загрузить вчерашний
и «откатиться» на предыдущее состояние

Что такое… версия?



Документы

черновик → на_проверку → финал

Игры

Сейв — можно загрузить вчерашний
и «откатиться» на предыдущее состояние

Программы

iOS 17 → 18

Office 2003 → 2010 → 2024

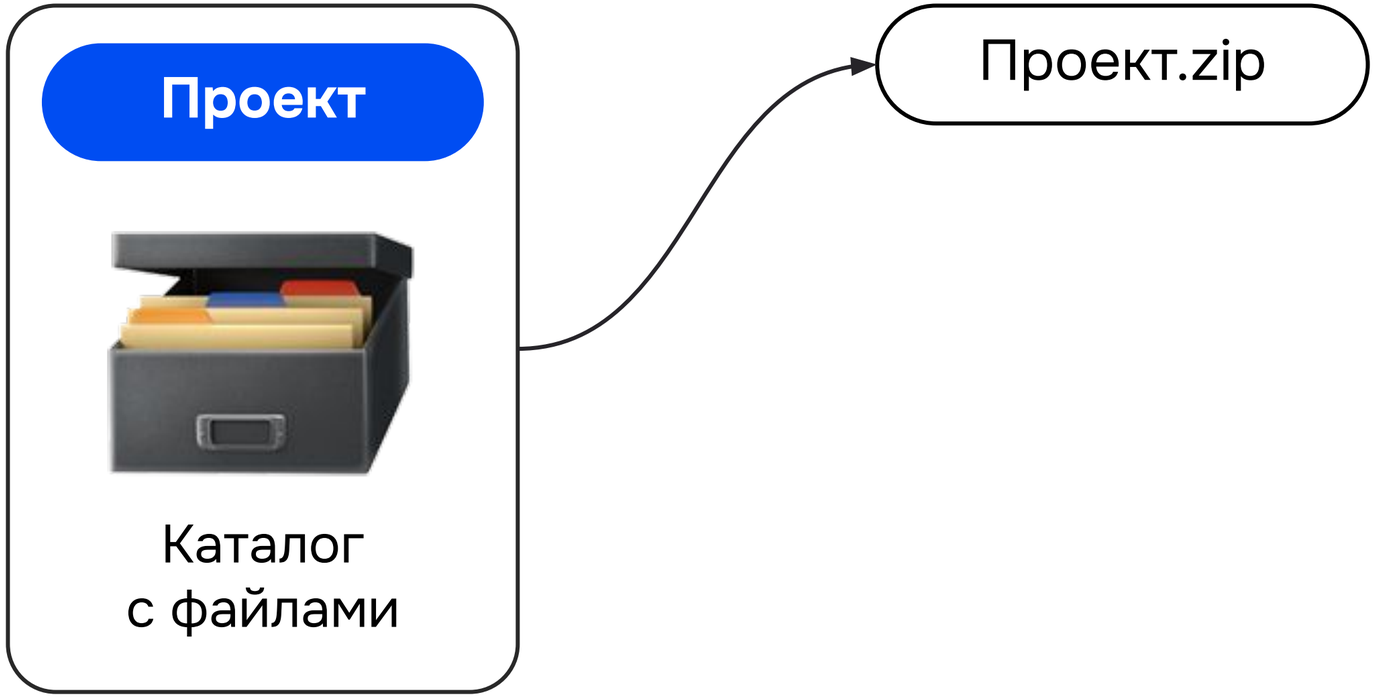
Что такое… версия?

Проект

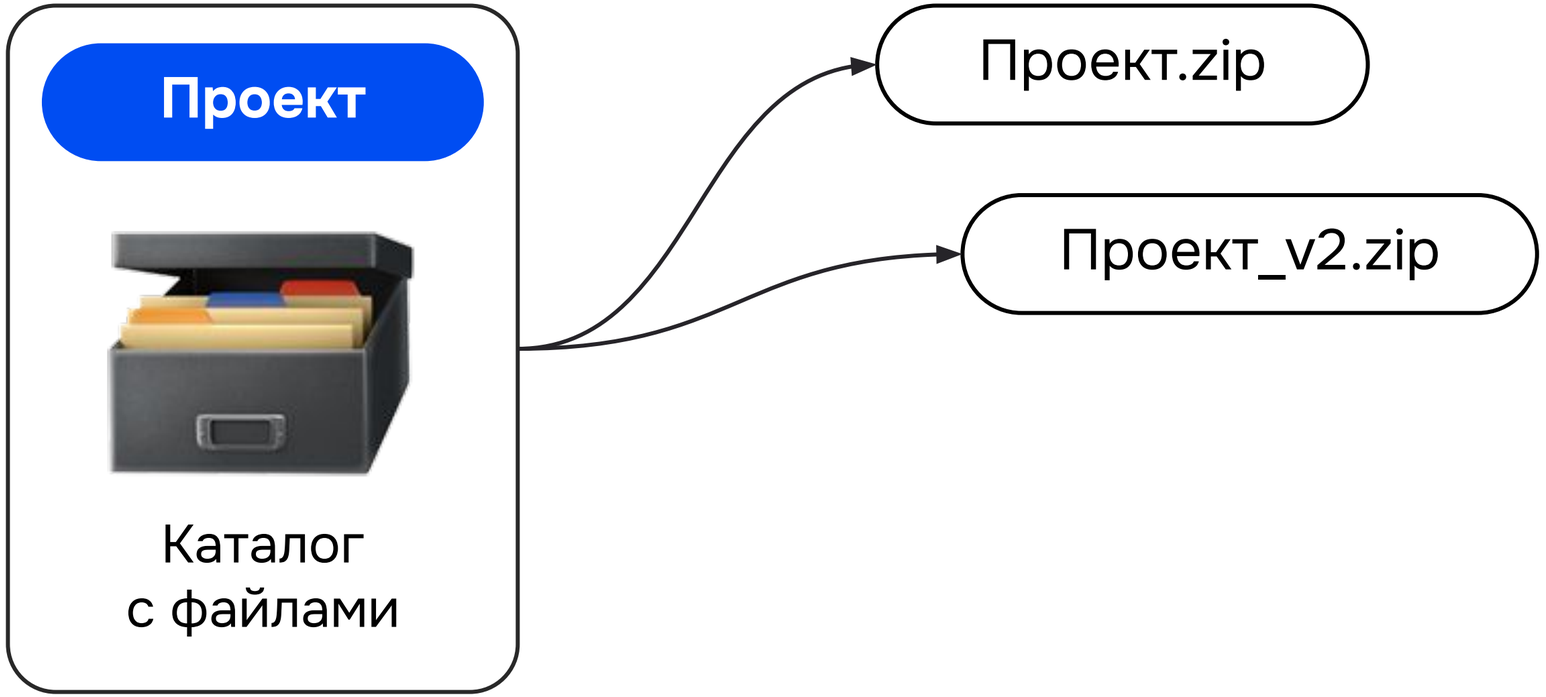


Каталог
с файлами

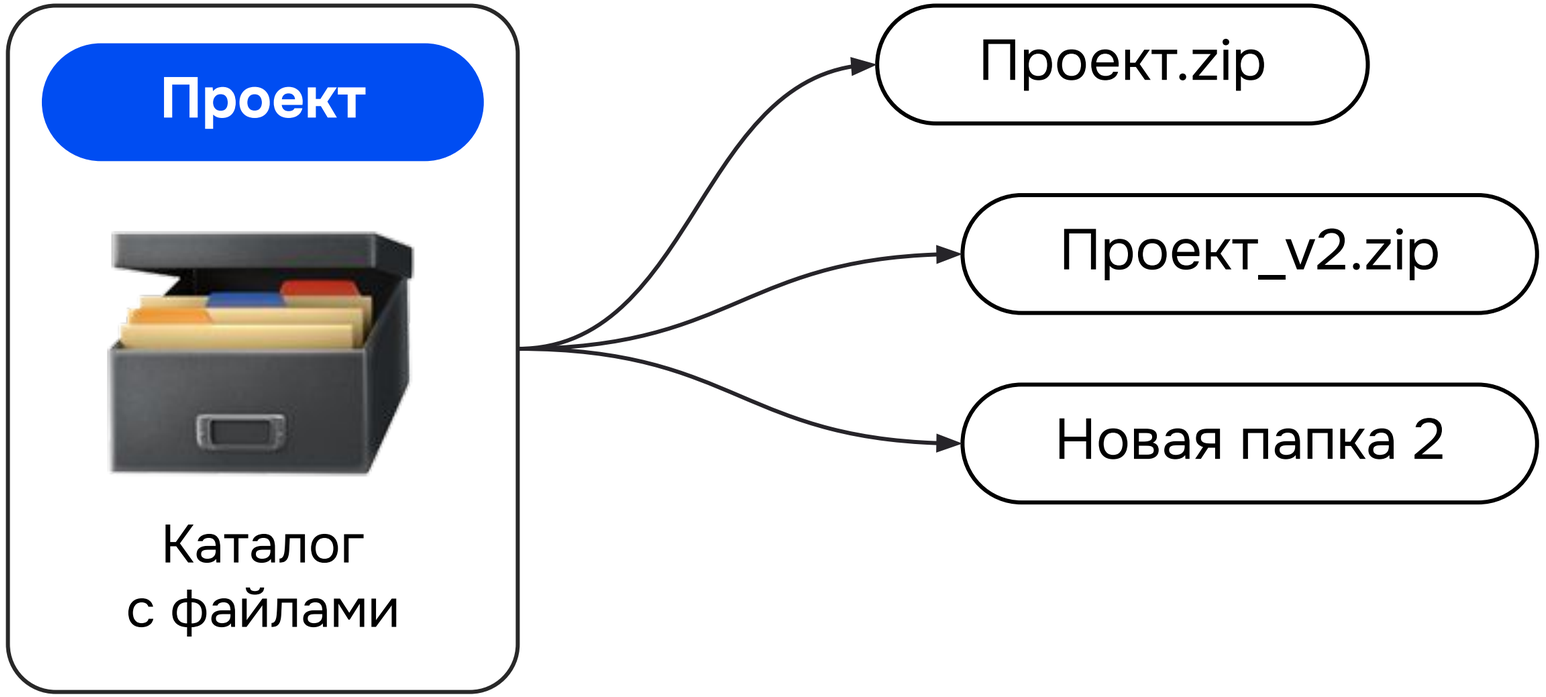
Что такое… версия?



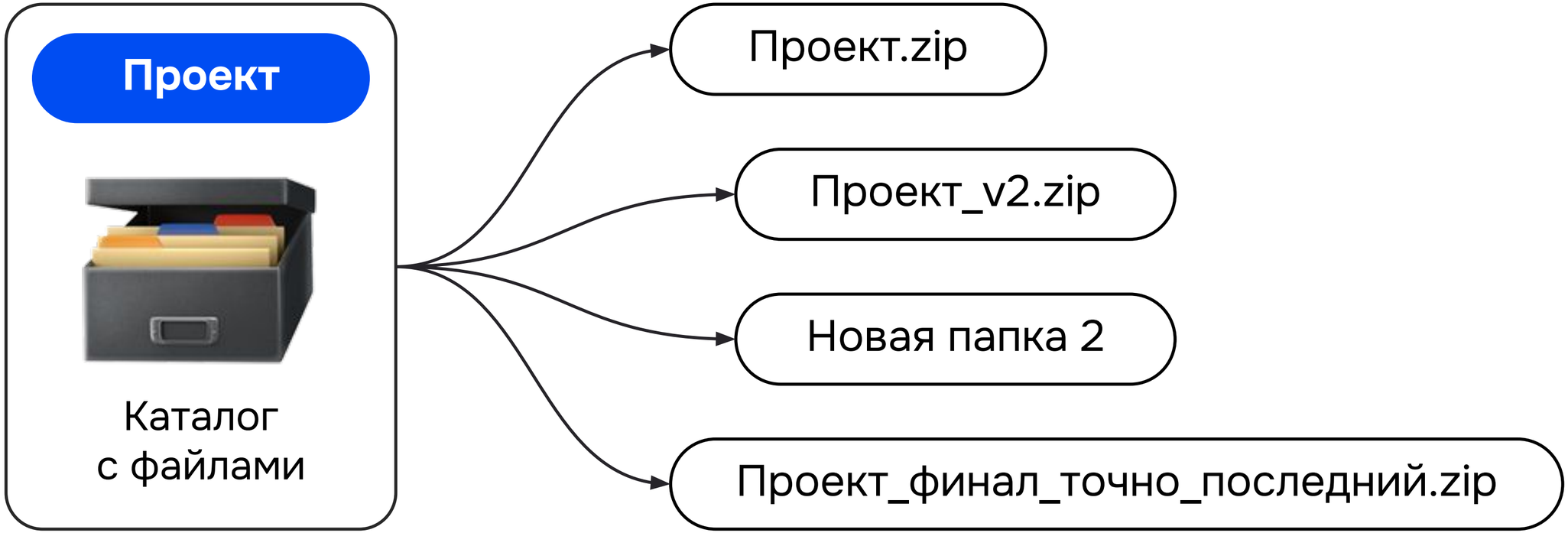
Что такое... версия?



Что такое... версия?



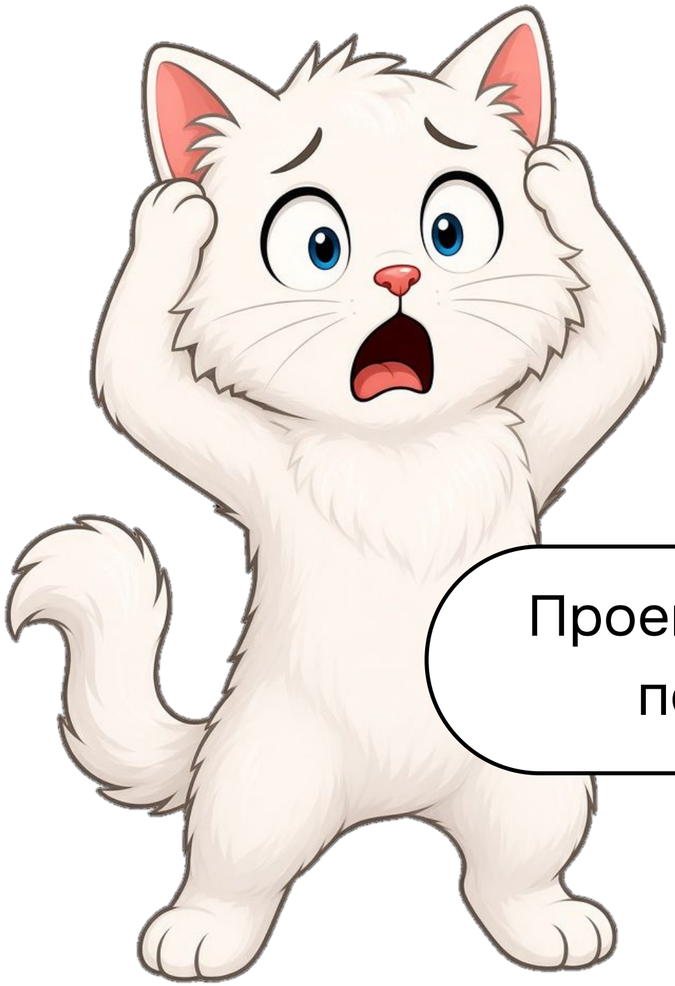
Что такое... версия?



Что такое Git?

Новая папка 2

Проект.zip



Проект_финал_точно_
последний.zip

Проект_v2.zip

Всем знакомая боль. К чему это приводит

01 «У меня работало вчера — что я поменял?»

Всем знакомая боль. К чему это приводит

01 «У меня работало вчера — что я поменял?»

02 «Как вдвоём править один и тот же файл?»

Что такое… версия?

Версия

Снимок (снапшот) чего-либо в конкретный момент времени



Системы контроля версий

От общего каталога до распределённой системы

Локальные

1970–80-е ▶ один компьютер, один человек

Локальные



1970–80-е ► один компьютер, один человек

- Source Code Control System (SCCS)
- Revision Control System (RCS)

Локальные

1970–80-е ► один компьютер, один человек

- Source Code Control System (SCCS)
- Revision Control System (RCS)

Централизованные

1980–2000-е ► один сервер — официальная правда

Локальные

1970–80-е ► один компьютер, один человек

- Source Code Control System (SCCS)
- Revision Control System (RCS)

Централизованные

1980–2000-е ► один сервер — официальная правда

- Concurrent Versions System (CVS)
- Subversion (SVN)



Локальные

1970–80-е ► один компьютер, один человек

- Source Code Control System (SCCS)
- Revision Control System (RCS)

Централизованные

1980–2000-е ► один сервер — официальная правда

- Concurrent Versions System (CVS)
- Subversion (SVN)

Распределённые

2000-е ► у каждого полная история

Локальные

1970–80-е ► один компьютер, один человек

- Source Code Control System (SCCS)
- Revision Control System (RCS)

Централизованные

1980–2000-е ► один сервер — официальная правда

- Concurrent Versions System (CVS)
- Subversion (SVN)

Распределённые

2000-е ► у каждого полная история

- BitKeeper
- Mercurial
- Git



Локальные

1970–80-е:
один компьютер, один человек

- Source Code Control System (SCCS)
- Revision Control System (RCS)



Централизованные

1980–2000-е:
один сервер — официальная правда

- Concurrent Versions System (CVS)
- Subversion (SVN)



Распределённые

2000-е:
у каждого полная история

- BitKeeper
- Mercurial
- Git



С чем хорошо и плохо работает Git

С чем хорошо работает Git

Текст — файл из строк символов.
Его можно открыть в обычном редакторе
и читать глазами

С чем хорошо работает Git

Текст — файл из строк символов.
Его можно открыть в обычном редакторе и читать глазами:

- код;
- Markdown;
- HTML;
- JSON;
- CSV;
- большинство конфигов — это текст.

Примеры текста

- Исходный код: .py .js .java .go
- Разметка и данные: .md .html .json .csv .yml
- Настройки проекта: .gitignore Dockerfile Makefile
- Обычный .txt

С чем Git работает плохо

Бинарник — не строки, а байты «для программы». Глазами, как текст, его не прочитать:

С чем Git работает плохо

Бинарник — не строки, а байты «для программы». Глазами, как текст, его не прочитать:

- картинки;
- видео;
- PDF;
- шрифты;
- архивы;
- собранные программы;
- базы вроде SQLite;
- Word и Excel (.docx, .xlsx).

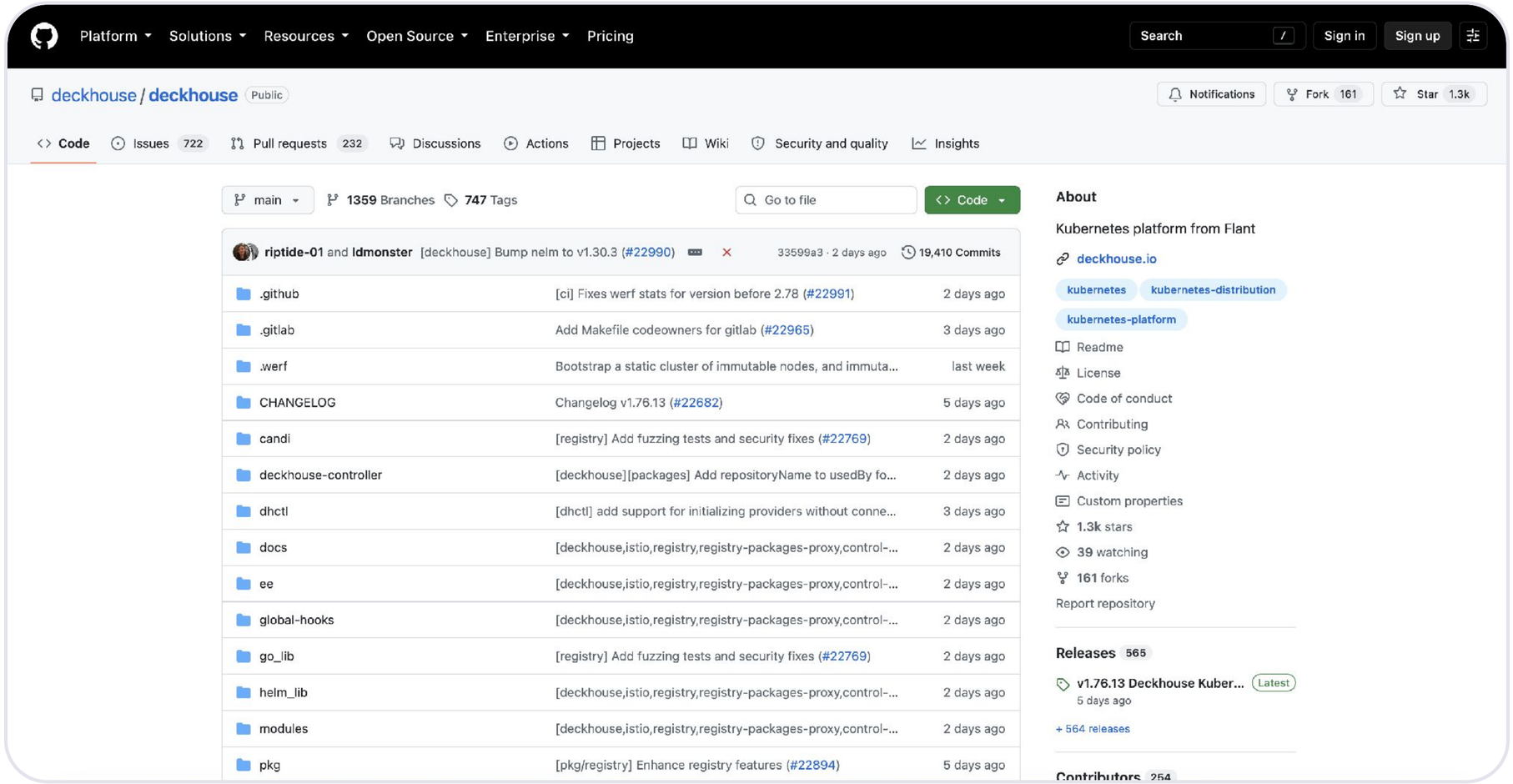
Примеры не текста

- Картинки и медиа: .png .jpg .mp4 .wav
- Документы программ: .pdf .docx .xlsx
- Архивы и сборка: .zip .exe .class node_modules
- Данные «для программы»: .sqlite шрифты модели

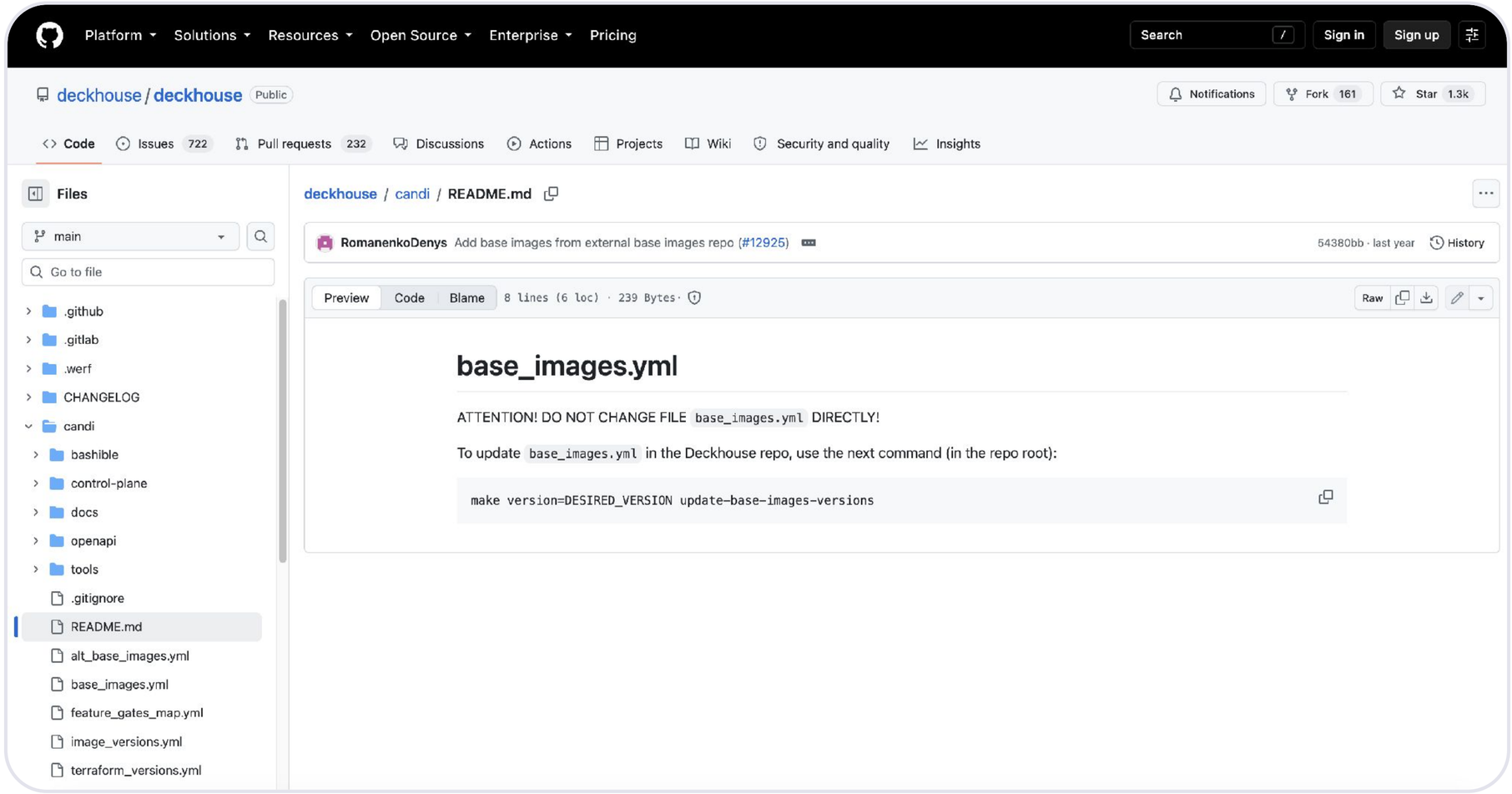
Git и GitHub

(GitLab, Gitea, etc...)

GitHub
и прочие



GitHub
и прочие



Git ≠ GitHub

программа и сайт

Git ≠ GitHub

программа и сайт

«Гит» и **«гитхаб»** звучат как **одно слово**. В разговоре часто так и говорят: «скинь в гит», «у меня на гитхабе».
Из-за этого кажется, что Git — это сайт.

На самом деле нет: **GitHub — это сервис**, хостинг кода, который отображает содержимое репозитория и позволяет выполнять некоторые задачи в более удобном формате.

Хостинг кода и его плюшки

Что добавляет хостинг

01 Место для remote

Общая копия, чтобы не носить проект на флешке

Что добавляет хостинг

01 Место для remote

Общая копия, чтобы не носить проект на флешке

02 Веб-интерфейс

Код, история и diff в браузере, комментарии

Что добавляет хостинг

01 Место для remote

02 Веб-интерфейс

03 Права доступа

Кто может читать репозиторий, кто писать. У локальной папки этого нет

Что добавляет хостинг

01 Место для remote

02 Веб-интерфейс

03 Права доступа

Кто может читать репозиторий, кто писать. У локальной папки этого нет

04 Механизм Codeowners

Оповещение «владельцев» конкретных файлов при событиях с ними

Что добавляет хостинг

01 Место для remote

02 Веб-интерфейс

03 Права доступа

04 Механизм Codeowners

05 Issues и Discussions

Запросы изменений и дискуссии
рядом с репозиторием

Что добавляет хостинг

01 Место для remote

02 Веб-интерфейс

03 Права доступа

04 Механизм Codeowners

05 Issues и Discussions

Запросы изменений и дискуссии рядом с репозиторием

06 CI

Автопроверки после push: тесты, линтеры

Что добавляет хостинг

01 Место для remote

02 Веб-интерфейс

03 Права доступа

04 Механизм Codeowners

05 Issues и Discussions

06 CI

07 Социальный слой

Чужие проекты, форки, звёзды

Git хранит версии

Хостинг помогает ими делиться
и обсуждать изменения

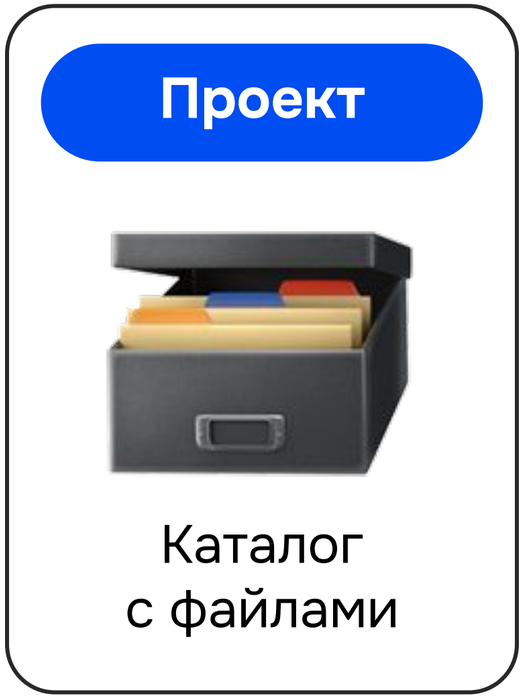


Базовые понятия



Репозиторий

Репозиторий — проект вместе с его историей.



Репозиторий

Репозиторий — проект вместе с его историей.

В каталоге проекта обычные файлы. Рядом, в скрытой папке .git, хранятся все данные об истории проекта.

Репозиторий

Репозиторий — проект вместе с его историей.

В каталоге проекта обычные файлы. Рядом, в скрытой папке .git, хранятся все данные об истории проекта.

Пока есть .git — это репозиторий.
Убрали .git — осталась просто папка с файлами, история исчезла.

Репозиторий

Репозиторий — проект вместе с его историей.

В каталоге проекта обычные файлы. Рядом, в скрытой папке .git, хранятся все данные об истории проекта.

Пока есть .git — это репозиторий.
Убрали .git — осталась просто папка с файлами, история исчезла.

Эту историю можно копировать целиком: на другой компьютер, на флешку, на сервер. В этом и есть **распределённость**.

Репозиторий

Локальный репозиторий — **на вашем компьютере**. С ним вы работаете каждый день: правите, коммитите, смотрите историю.



Репозиторий

Локальный репозиторий — **на вашем компьютере**. С ним вы работаете каждый день: правите, коммитите, смотрите историю.



Удалённый (remote) — ещё одна **копия где-то**: GitHub, GitLab, свой сервер, даже папка на другом диске.



Репозиторий





Они не обязаны совпадать каждую секунду. Вы коммитите **локально**, **без сети**. Когда готовы — отправляете снимки на remote-копию.

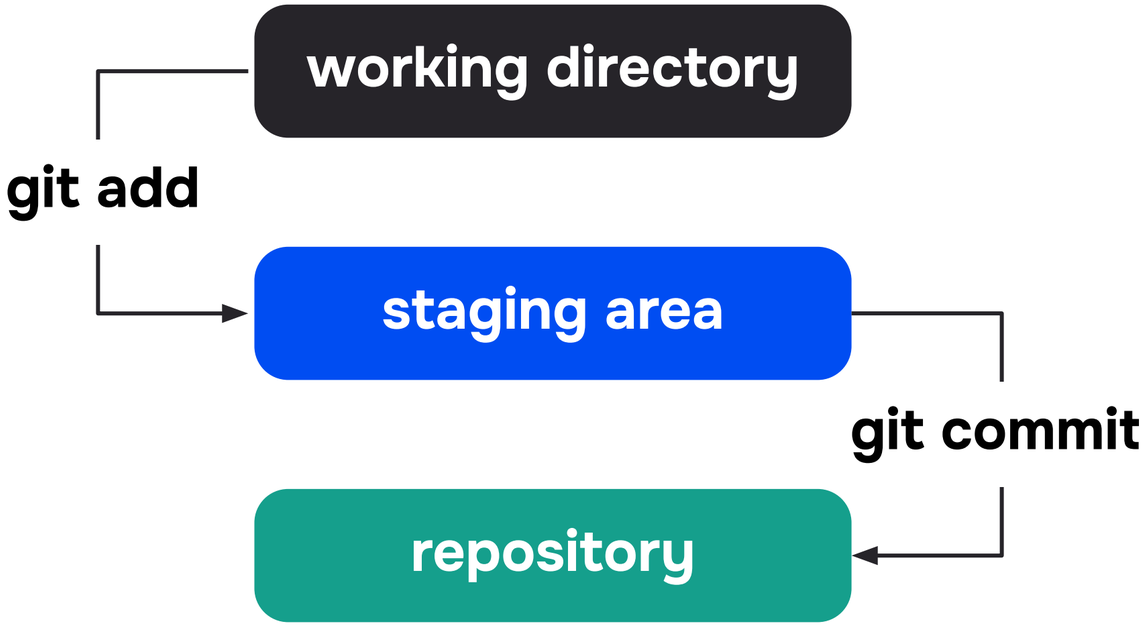
Working directory

Working directory — это **каталог**, в котором лежат файлы проекта и история Git.



Staging area, index

Staging area — это **промежуточный этап** между изменением файлов в рабочей директории и фиксацией этих изменений в истории проекта (коммитом).



Коммит



Commit — сохранённая версия.

Коммит — **снимок всего проекта** в конкретный момент плюс короткое объяснение, зачем этот снимок сделали (комментарий).

Это не «сохранить файл», как в Word. Это **«запомнить состояние репозитория»** вместе с автором, датой и сообщением.

Коммиты связаны: у каждого есть родитель — предыдущий снимок. Цепочка и есть история.

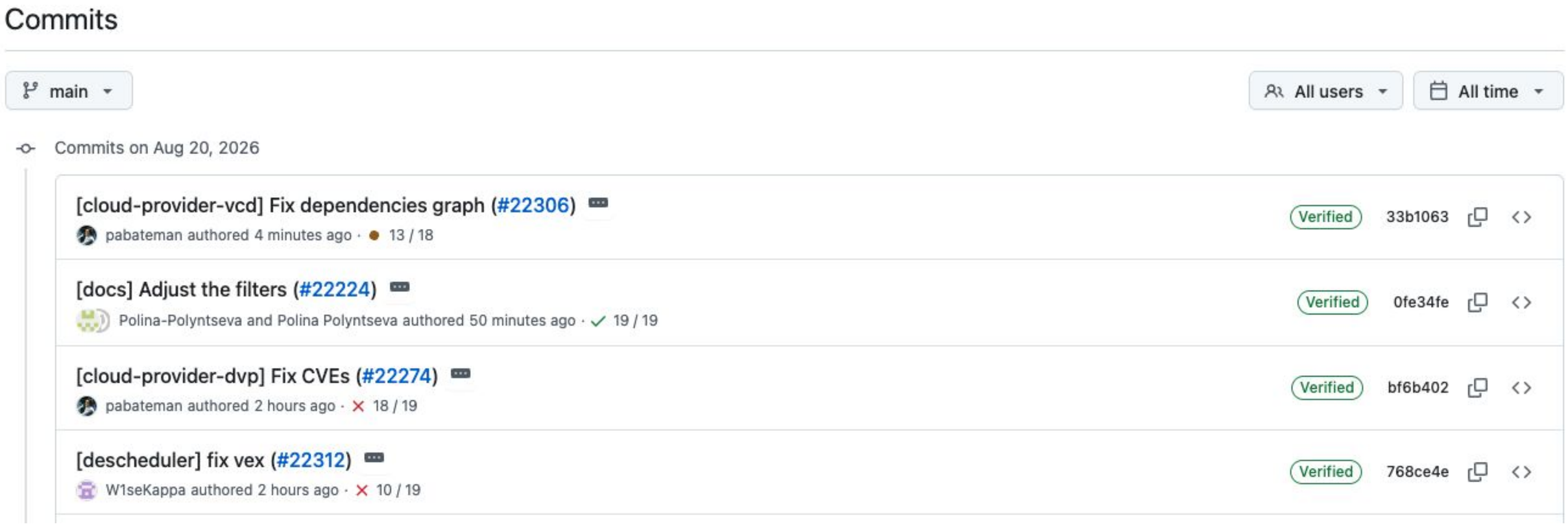
Сообщение коммита

Сообщение читают люди: через месяц вы сами, коллеги и тот, кто ищет, когда сломалось.

Хороший тон — писать, что изменилось и зачем. «Исправить вход, если пароль с пробелом» лучше, чем fix или asdasd.

Плохое сообщение не ломает Git. Оно ломает историю как рассказ. Потом невозможно понять версии.

Сообщение коммита



История — это цепочка

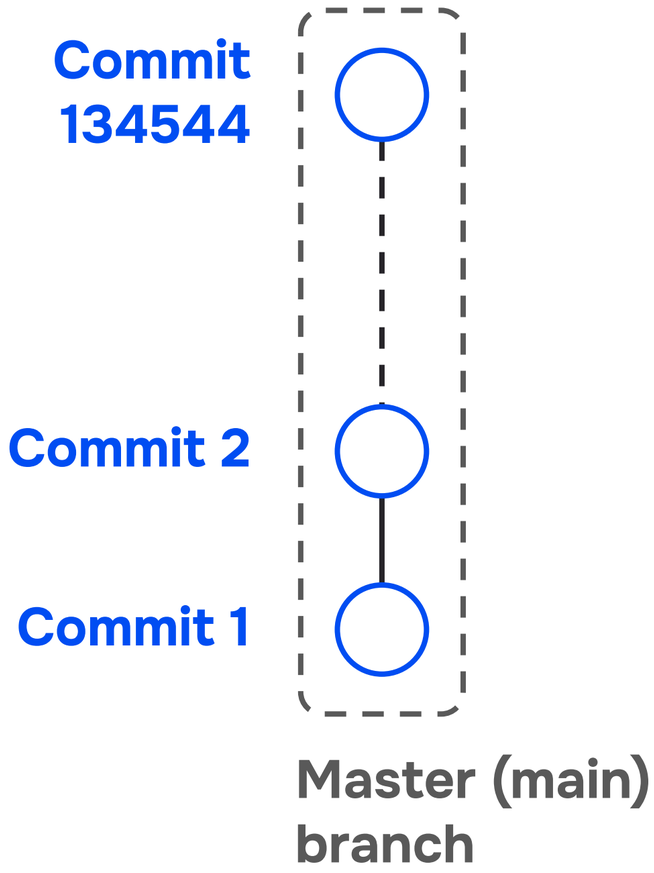


Один коммит бесполезен сам по себе.
Сила в том, что снимки выстроены по времени:
было → **стало** → **стало ещё**.

Каждый новый коммит знает, из какого появился,
поэтому Git может показать разницу между
любыми двумя точками.

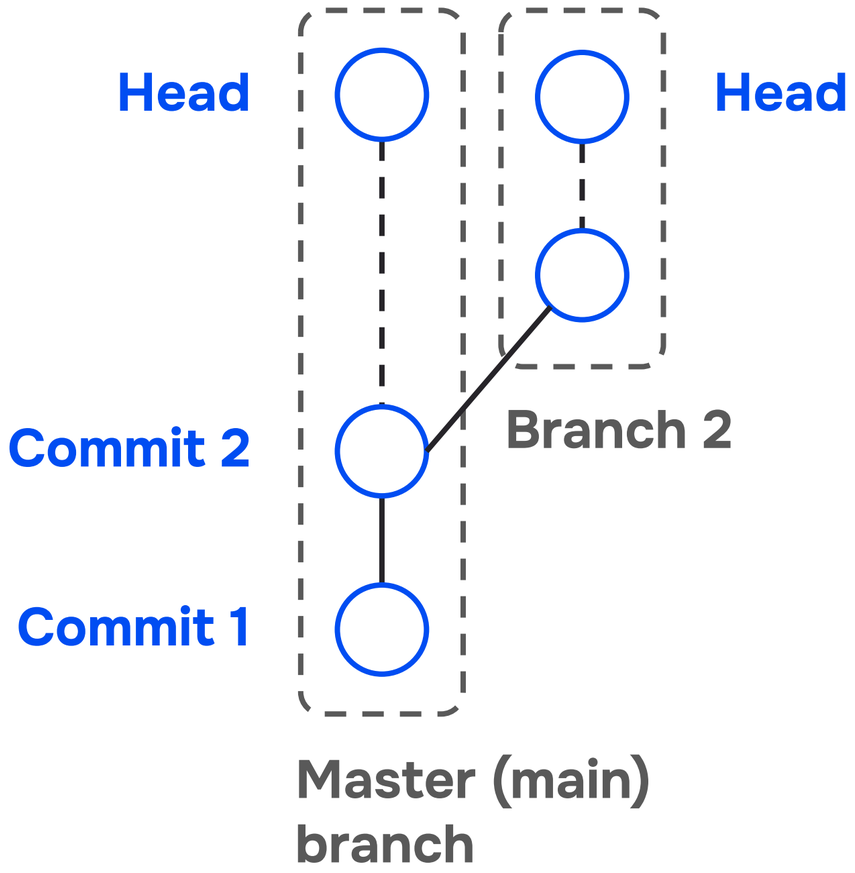
Ветка (branch)

Ветка — **цепочка** связанных коммитов.



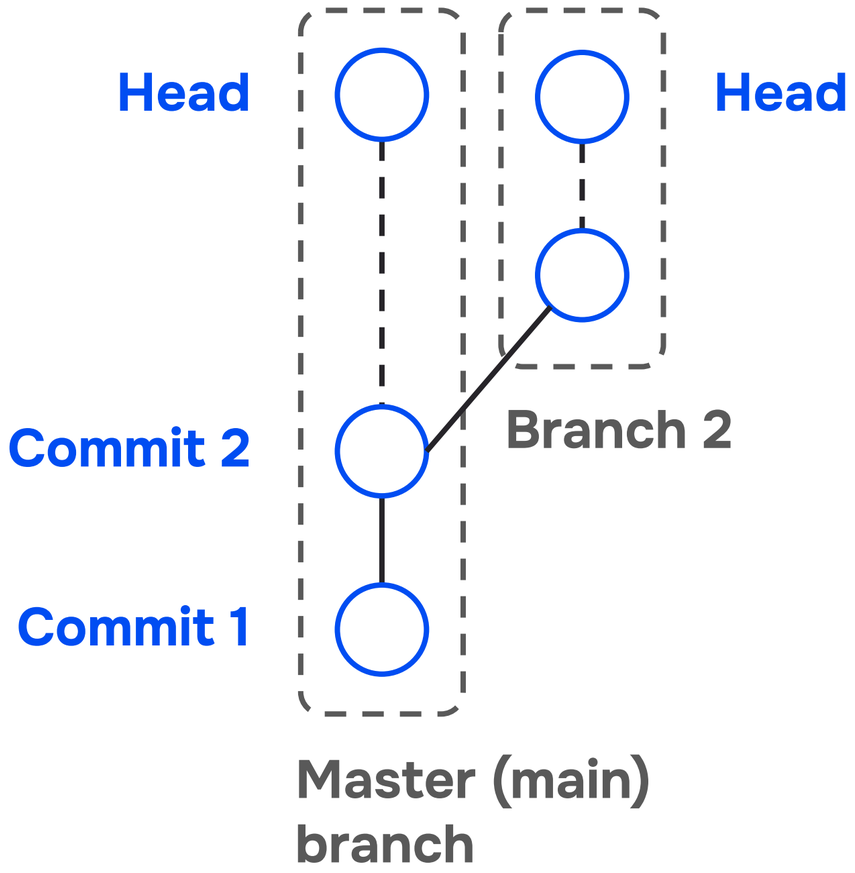
HEAD

HEAD — **указатель** на самый последний коммит в ветке.



Зачем нужны ветки

- Чтобы **не мешать стабильному коду**. Новая фича, срочный фикс, сомнительный эксперимент — отдельная ветка.
- Сломалось в эксперименте — **основная ветка не пострадала**. Можно выбросить сломанную ветку или потом слить удачное.



Remote — удалённый репозиторий

Remote — **имя другой копии репозитория**, обычно на сервере.
Не «облако Git», а адрес, куда можно отправить историю
и откуда её взять.

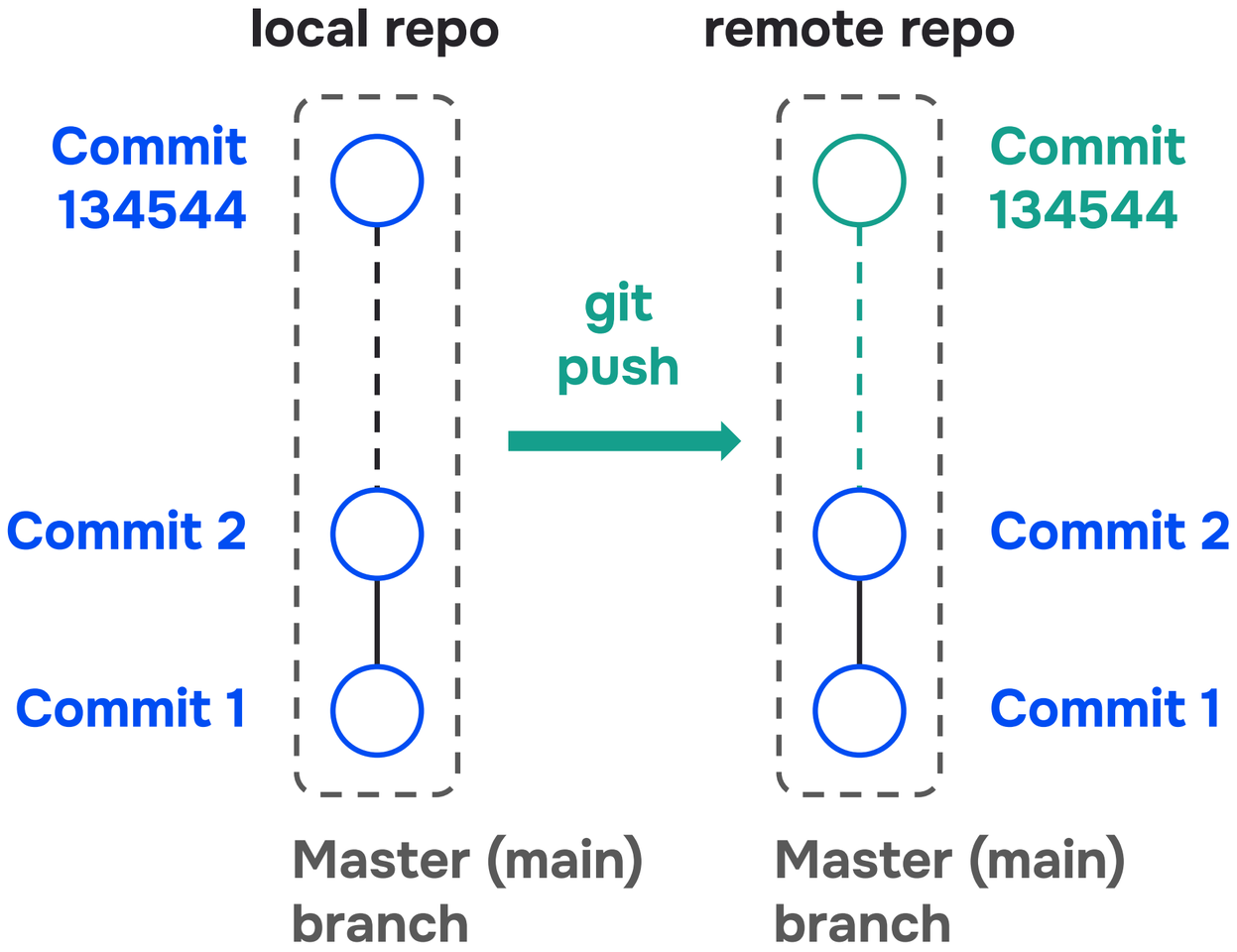
Самое распространённое имя — **origin**: тот remote, откуда клонировали, или тот, который считаем основным.

Clone

Clone — **скачать** чужой репозиторий себе целиком:
файлы, история, ветки.

Push и pull

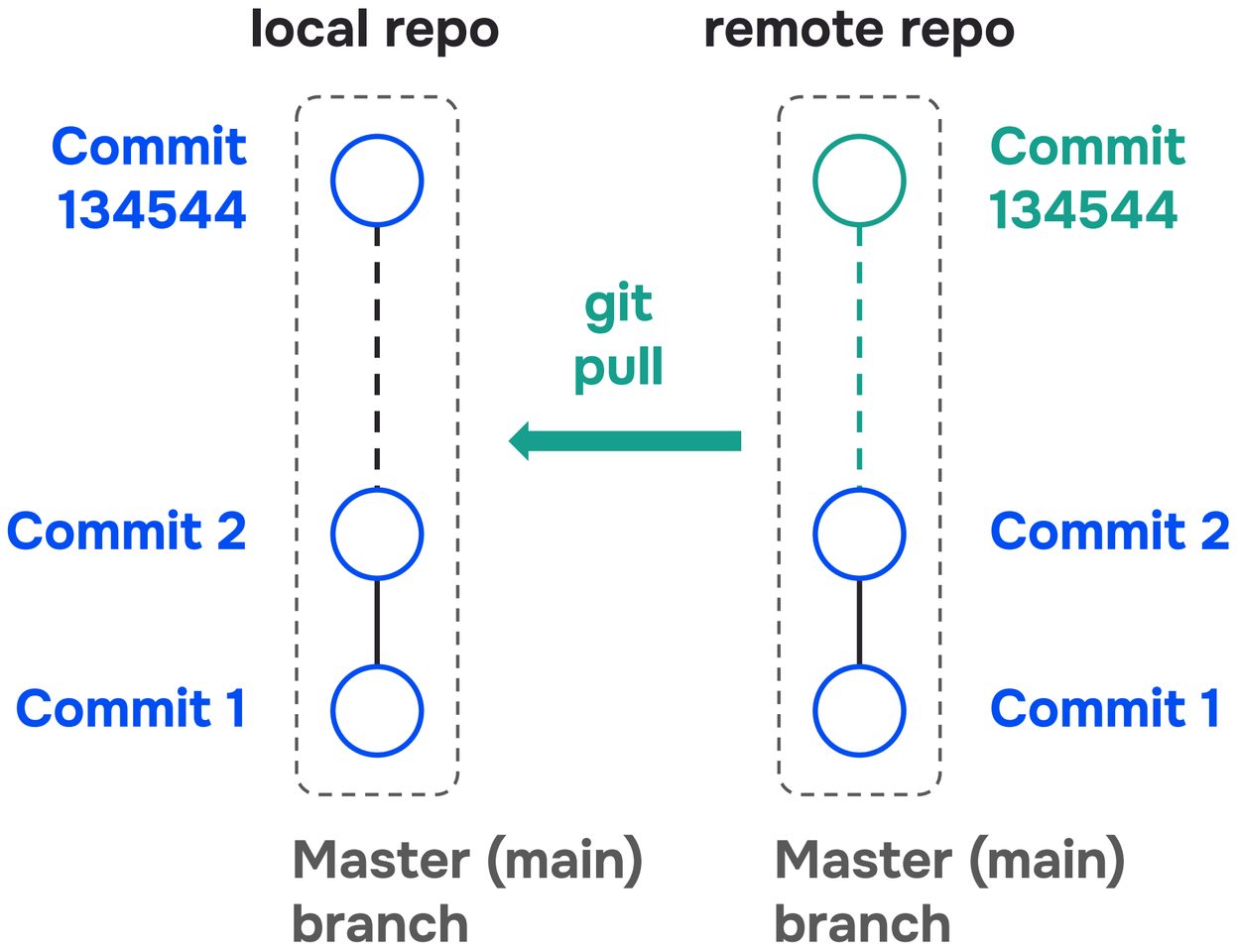
Push — **отправить** свои коммиты на remote.
Пока не сделали push, их видите только вы.



Push и pull

Push — **отправить** свои коммиты на remote. Пока не сделали push, их видите только вы.

Pull — **забрать** чужие коммиты к себе. «Под капотом» Git сначала скачивает, потом объединяет.



Git log — история коммитов

Log — история снимков: кто, когда, с каким сообщением.

```
commit 4640d8f8224fe7b5c7dd54bf71cc4a558a5940bf (HEAD -> main, origin/main,
origin/HEAD)
Author: Artem Kuleshov <41912276+Fkuloff@users.noreply.github.com>
Date:   Thu Sep 17 15:36:48 2026 +0300

    [deckhouse] cover the Module package controller with reconciler tests
(#22816)

    Signed-off-by: Artem Kuleshov <artem.kuleshov@flant.com>
    Signed-off-by: Pavel Okhlopkov <pavel.okhlopkov@flant.com>
    Co-authored-by: Pavel Okhlopkov <pavel.okhlopkov@flant.com>

commit 1afd1e78fcd4459794b86a69263533ce70d693c7
Author: Dmitry Trofimov <dmitry.trofimov@flant.com>
Date:   Thu Sep 17 15:11:04 2026 +0300

    [control-plane-manager] Fix doubled usage in autotune, add 10% request
margin (#22910)

    Signed-off-by: Dmitry Trofimov <dmitry.trofimov@flant.com>
    Signed-off-by: Nikolay Gorbatov <nikolay.gorbatov@flant.com>
    Co-authored-by: Nikolay Gorbatov <nikolay.gorbatov@flant.com>

commit 6bb7c5bd4783a01596cc227903e7a9079a88e63a
Author: Artem Kladov <6360800+z9r5@users.noreply.github.com>
Date:   Thu Sep 17 14:26:58 2026 +0300

    [docs] Update product link references (#23135)

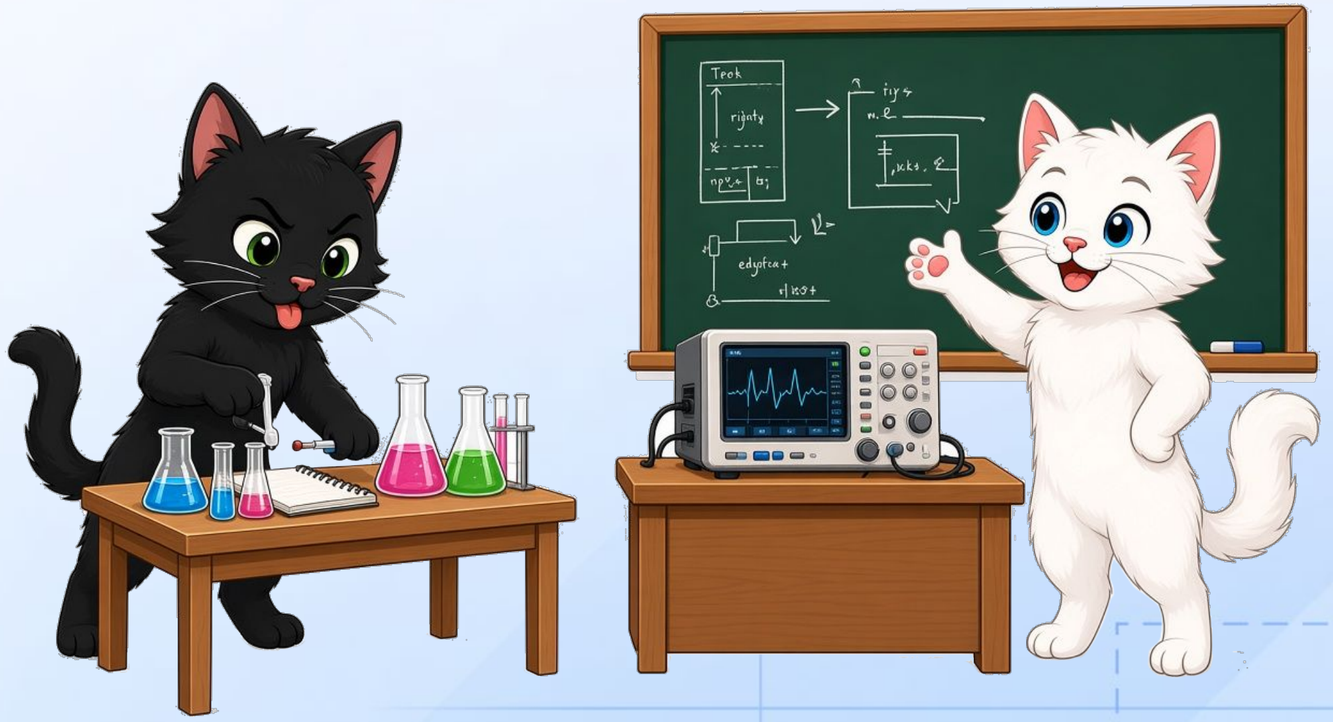
    Signed-off-by: Artem Kladov <artem.kladov@flant.com>
```

Git diff — что изменилось

Diff показывает, что именно изменилось в тексте: какие строки ушли, какие пришли.

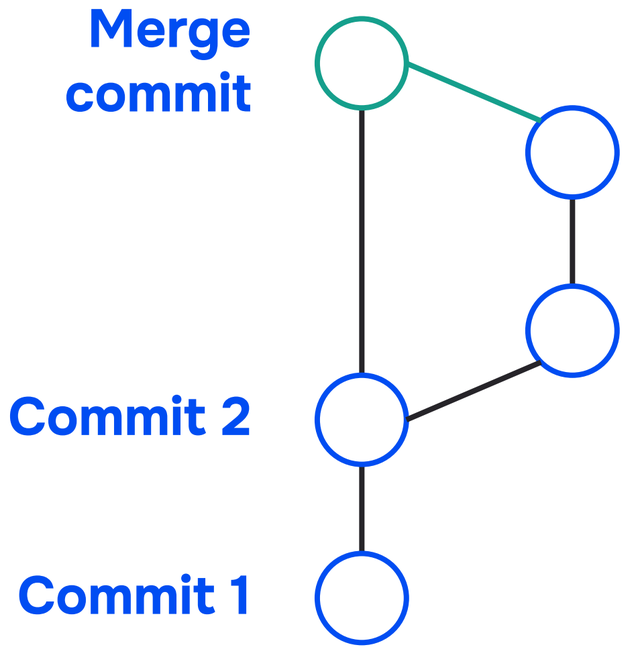
```
diff --git a/modules/000-common/images/init/werf.inc.yaml
b/modules/000-common/images/init/werf.inc.yaml
index 2a452d04ce..5ea67a04c8 100644
--- a/modules/000-common/images/init/werf.inc.yaml
+++ b/modules/000-common/images/init/werf.inc.yaml
@@ -1,21 +1,33 @@
-{{- $binaries := "/bin/sh /bin/chown /bin/chmod /bin/ls /bin/cp /bin/rm
/bin/mkdir /bin/nc /bin/sleep /bin/true" }}
 ---
 image: {{ .ModuleName }}/{{ .ImageName }}
-fromImage: common/distroless
+from: common/distroless
 import:
 - image: {{ .ModuleName }}/{{ .ImageName }}-artifact
-  add: /relocate
+  add: /
   to: /
   before: setup
+  includePaths:
+- image: registrypackages/netcat-artifact
+  before: setup
 {{- include "image mount points" . }}
 ---
 image: {{ .ModuleName }}/{{ .ImageName }}-artifact
-fromImage: common/relocate-artifact
+from: {{ index $.Images "builder/distroless" }}
 final: false
 shell:
   beforeInstall:
-  {{- include "alt packages proxy" . | nindent 2 }}
-  - apt-get install -y netcat
-  - find /var/lib/apt/ /var/cache/apt/ -type f -delete
```

Продвинутое использование



Merge

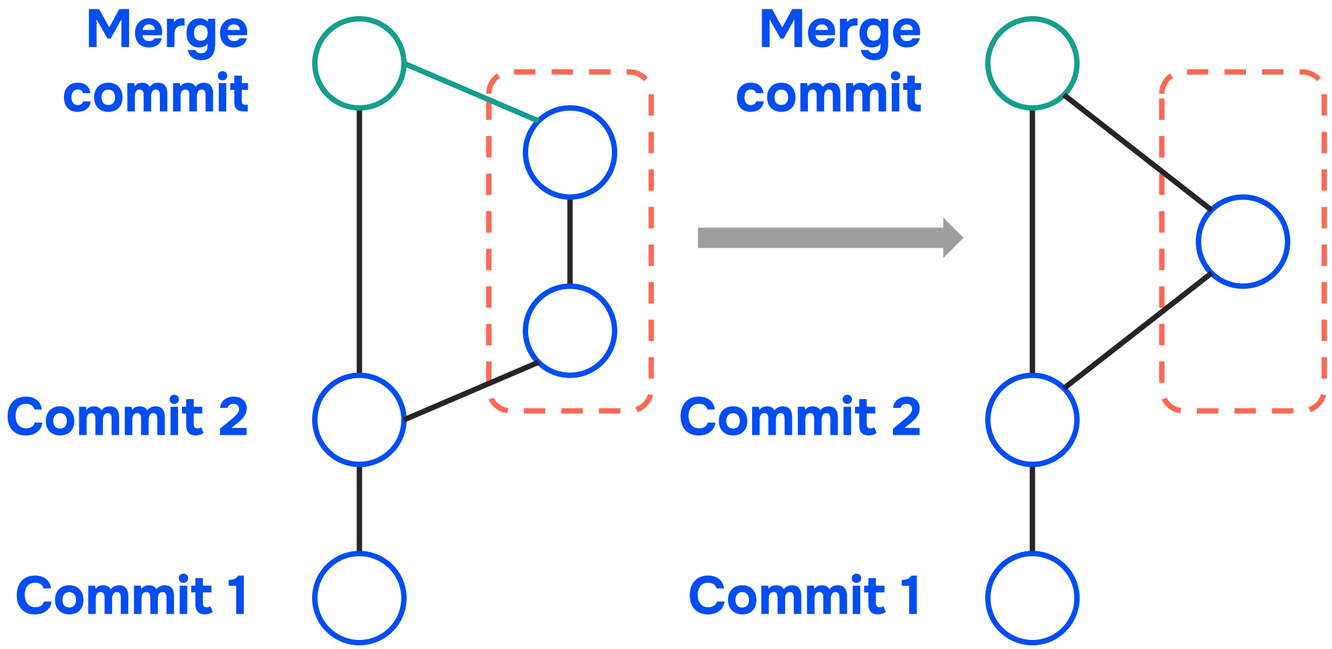
Две ветки встречаются. Git пишет особый коммит: «здесь мы соединились».
Это merge commit.



Squash

Много снимков ветки → один коммит на main

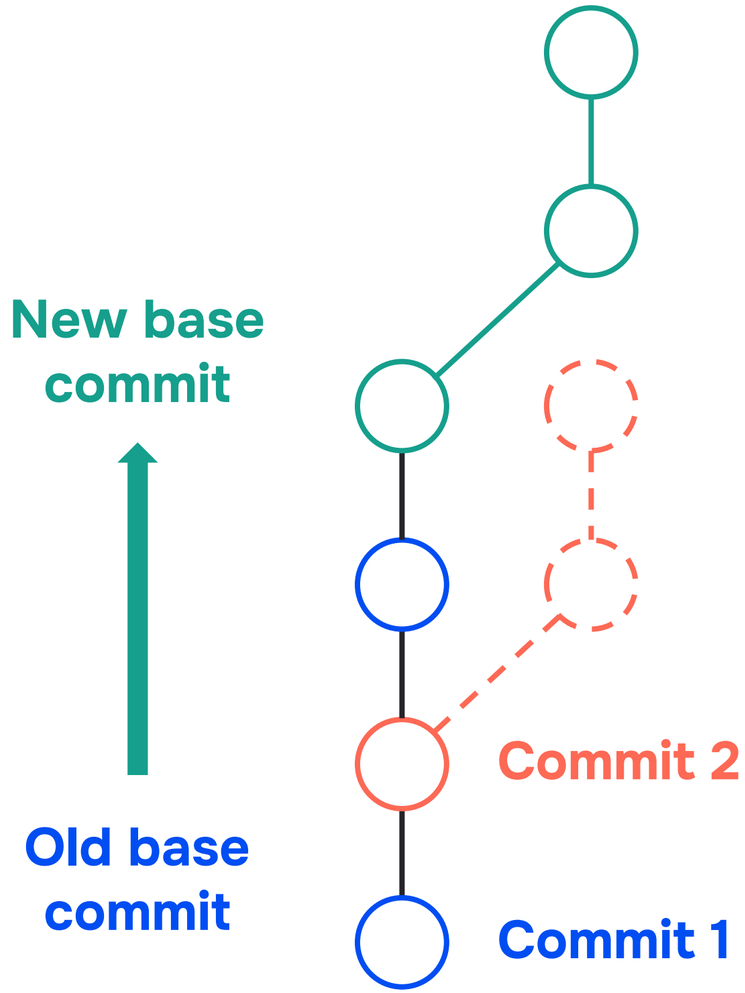
На ветке часто куча черновиков: wip, fix, «ещё раз», «теперь точно». Squash схлопывает эти снимки в один.



Rebase

Переставить свою ветку
на HEAD main

Коммиты как будто отрывают
от старого места и кладут поверх
свежего main.



--force

--force — насильное изменение.

Обычный push дописывает ваши новые снимки на сервер. Если там уже есть коммиты, которых нет у вас, Git скажет «нет»: иначе вы затрёте чужую работу.

Force push — «замени историю на сервере моей, несмотря ни на что».

Push Pull delete checkout Etc...

Утилиты для управления Git на компьютере

Как можно управлять гитом

01 **Терминал:** полный контроль, одни и те же команды на любой машине.

konstantin — man git — man — less ◂ man git — 186×49

```
GIT(1)                                                                  Git Manual                                                                  GIT(1)

NAME
       git - the stupid content tracker

SYNOPSIS
       git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
           [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
           [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
           <command> [<args>]

DESCRIPTION
       Git is a fast, scalable, distributed revision control system with an unusually rich command set that provides both high-level operations and full access to internals.

       See gittutorial(7) to get started, then see giteveryday(7) for a useful minimum set of commands. The Git User's Manual[1] has a more in-depth introduction.

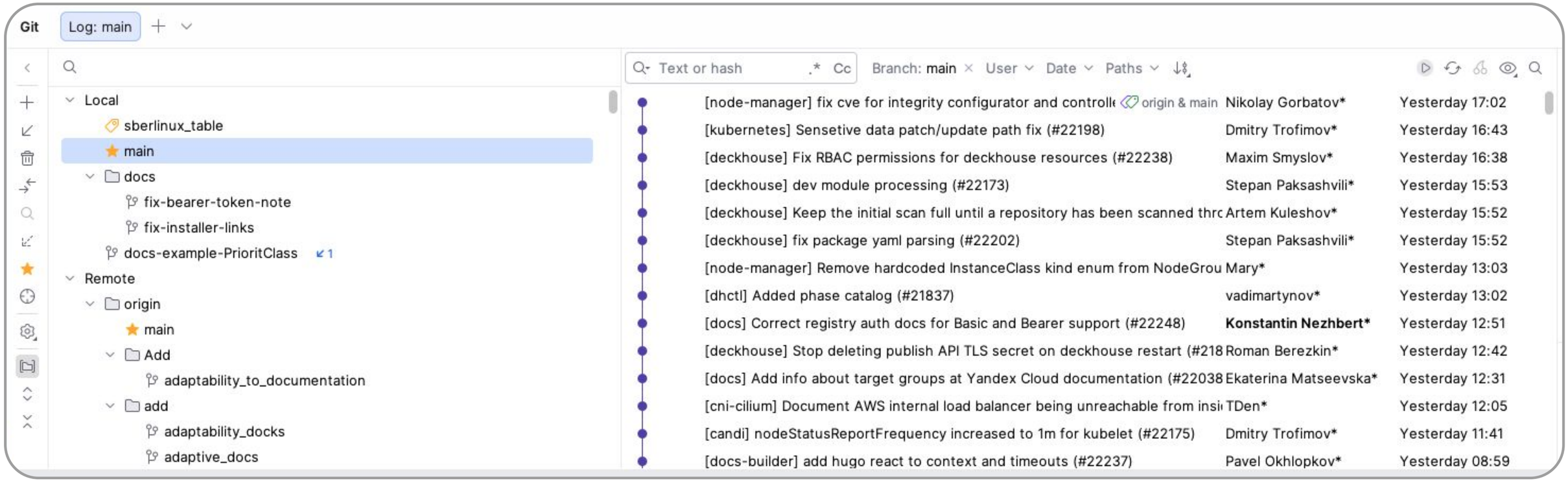
       After you mastered the basic concepts, you can come back to this page to learn what commands Git offers. You can learn more about individual Git commands with "git help
       command". gitcli(7) manual page gives you an overview of the command-line command syntax.

       A formatted and hyperlinked copy of the latest Git documentation can be viewed at https://git.github.io/htmldocs/git.html or https://git-scm.com/docs.

OPTIONS
       -v, --version
           Prints the Git suite version that the git program came from.
```

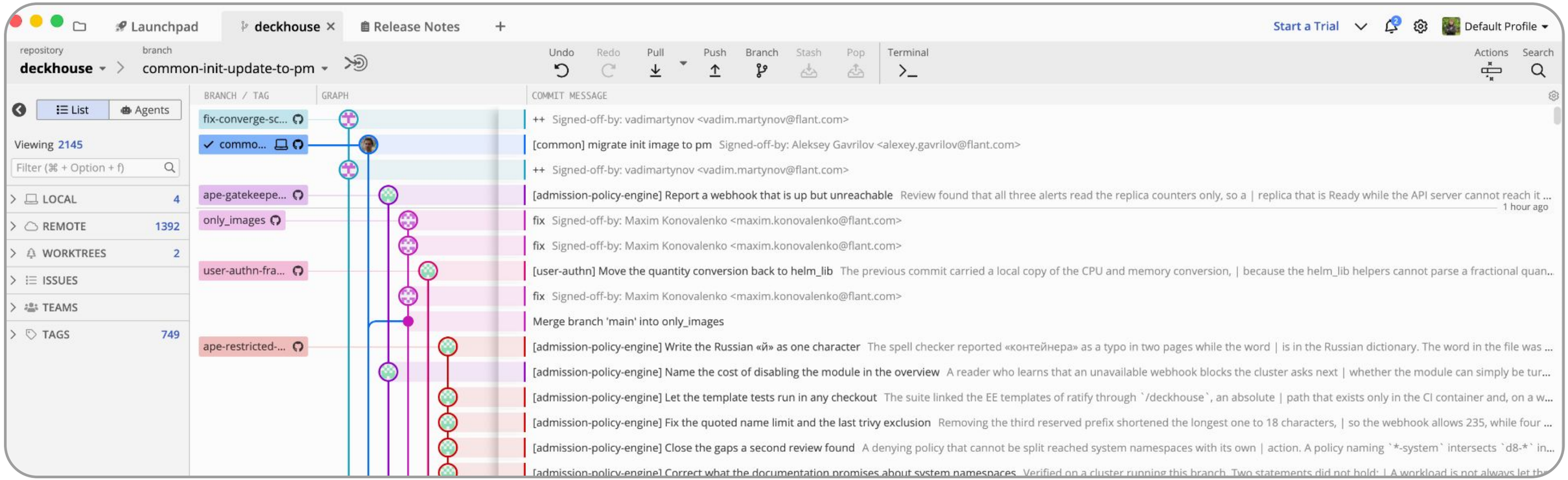
Как можно управлять гитом

02 **GUI в редакторе:** IDEA, PyCharm, Android Studio, VS Code, Cursor.



Как можно управлять гитом

03 **GUI в редакторе:** IDEA, PyCharm, Android Studio, VS Code, Cursor.



Минимальный рабочий цикл

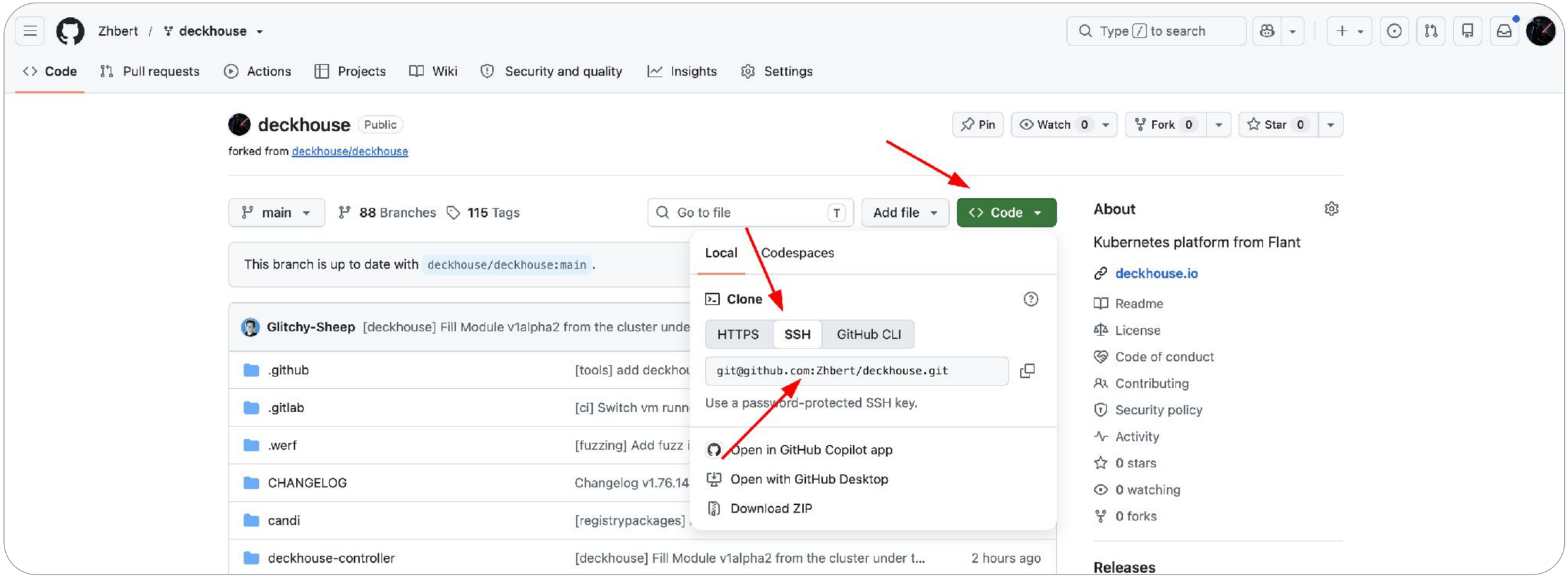
День из жизни новичка



Замени на главной странице документации название



01 Клонирование репозитория



01 Клонирование репозитория

git clone git@github.com:owner/deckhouse.git

```
$ git clone git@github.com:Zhbert/deckhouse.git
Cloning into 'deckhouse'...
remote: Enumerating objects: 310807, done.
remote: Counting objects: 100% (1056/1056), done.
remote: Compressing objects: 100% (802/802), done.
remote: Total 310807 (delta 414), reused 254 (delta 254), pack-reused 309751
(from 3)
Receiving objects: 100% (310807/310807), 412.73 MiB | 10.28 MiB/s, done.
Resolving deltas: 100% (214345/214345), done.
Updating files: 100% (15363/15363), done.
```

02 Новая ветка

git checkout -b update-docs

```
$ git checkout -b update-docs
Switched to a new branch 'update-docs'
```

03 Внесение изменений

M↓ OVERVIEW_RU.md ×

```
---
title: Документация Deckhouse Platform, пример для Стачки
permalink: ru/
description: Документация Deckhouse Platform.
rootLLMStxtURL: /products/kubernetes-platform/documentation/v1/llms.txt
lang: ru
---

{% capture anchors %}
%D0%BE%D1%81%D0%BD%D0%BE%D0%B2%D1%8B-%D0%BA%D0%BE%D0%BD%D1%84%D0%B8%D0%B3%D1%83%D1%80%D0%B0%D1%86%
 %D0%B8%D0%B7%D0%BC%D0%B5%D0%BD%D0%B5%D0%BD%D0%B8%D0%B5-%D0%BA%D0%BE%D0%BD%D1%84%D0%B8%D0%B3%D1%83%
 %B8-%D0%BA%D0%BB%D0%B0%D1%81%D1%82%D0%B5%D1%80%D0%B0,
 %D0%BD%D0%B0%D1%81%D1%82%D1%80%D0%BE%D0%B9%D0%BA%D0%B0-%D0%BC%D0%BE%D0%B4%D1%83%D0%BB%D1%8F,
```

04 Смотрим статус

git status

```
$ git status
On branch update-docs
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
  modified:   docs/documentation/pages/OVERVIEW_RU.md

no changes added to commit (use "git add" and/or "git commit -a")
```

04 Смотрим статус

git diff

```
$ diff --git a/docs/documentation/pages/OVERVIEW_RU.md
b/docs/documentation/pages/OVERVIEW_RU.md
index 4711818e8a..54b5835a81 100644
--- a/docs/documentation/pages/OVERVIEW_RU.md
+++ b/docs/documentation/pages/OVERVIEW_RU.md
@@ -1,5 +1,5 @@
 ---
-title: Документация Deckhouse Platform
+title: Документация Deckhouse Platform, пример для Стачки
 permalink: ru/
 description: Документация Deckhouse Platform.
 rootLLMStxtURL: /products/kubernetes-platform/documentation/v1/llms.txt
```

05

Добавляем изменения в staging

git add -A | git status

```
$ git add -A
$ git status
On branch update-docs
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
  modified:   docs/documentation/pages/OVERVIEW_RU.md
```

06 Коммитим изменения

git commit -m “Update title of the main page”

```
$ git commit -m "Update title of the main page"
1
[update-docs 5b7d07126d] Update title of the main page
 1 file changed, 1 insertion(+), 1 deletion(-)
```

07 Отправляем изменения изменения

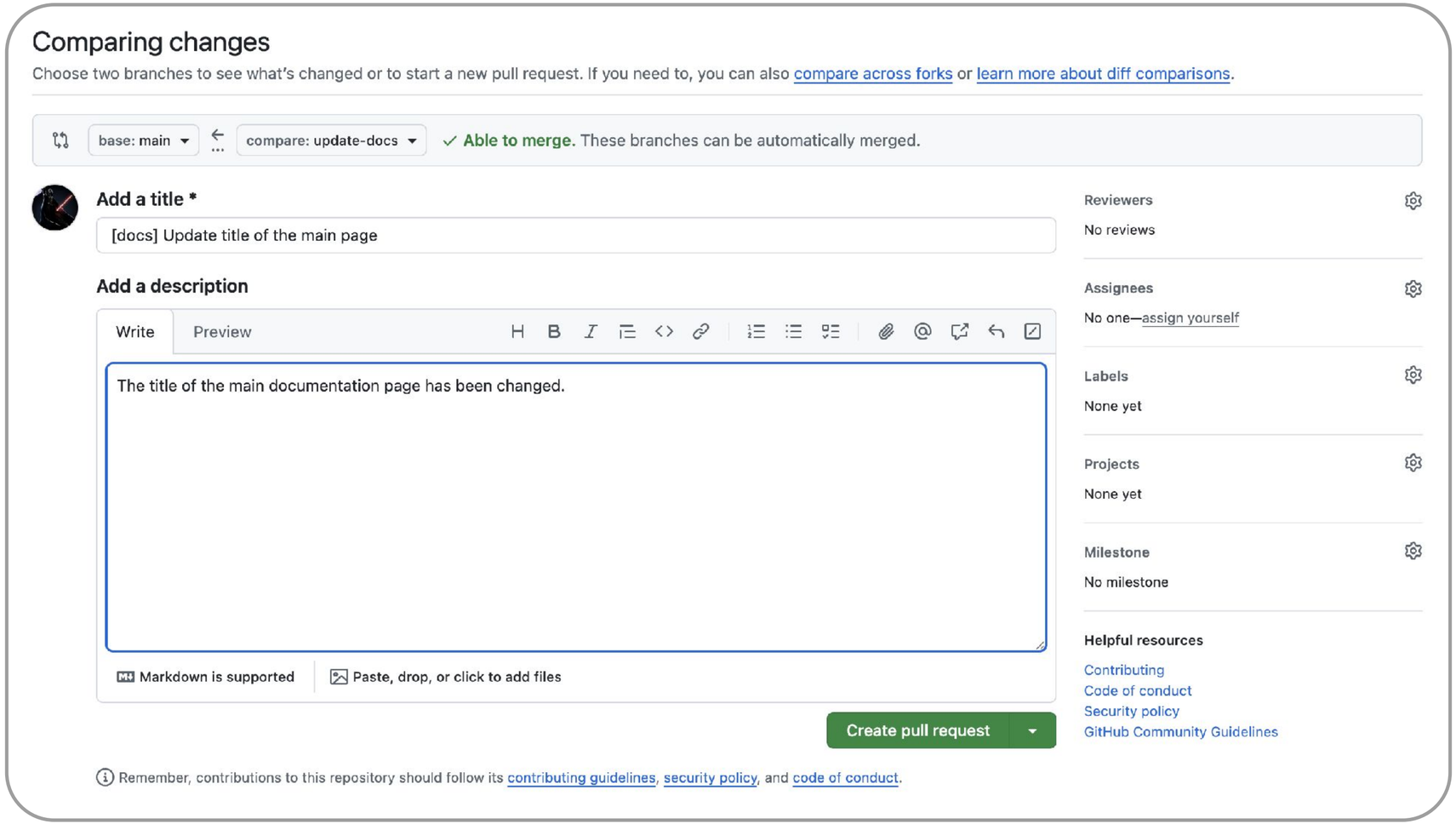
git push origin update-docs

```
$ git push origin update-docs

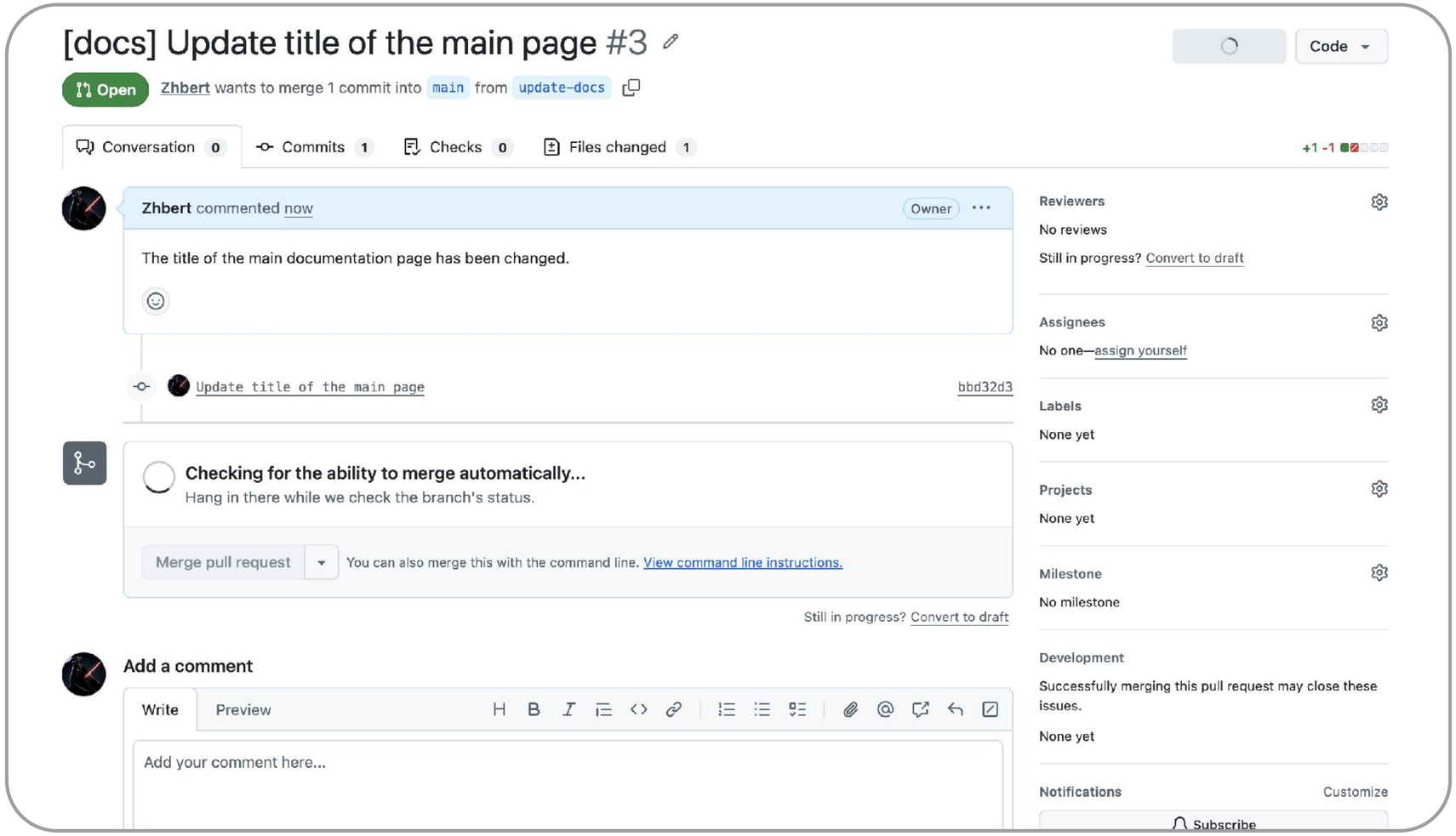
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 10 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 553 bytes | 553.00 KiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
remote:
remote: Create a pull request for 'update-docs' on GitHub by visiting:
remote:      https://github.com/Zhbert/deckhouse/pull/new/update-docs
remote:
To github.com:Zhbert/deckhouse.git
 * [new branch]            update-docs -> update-docs
```

08

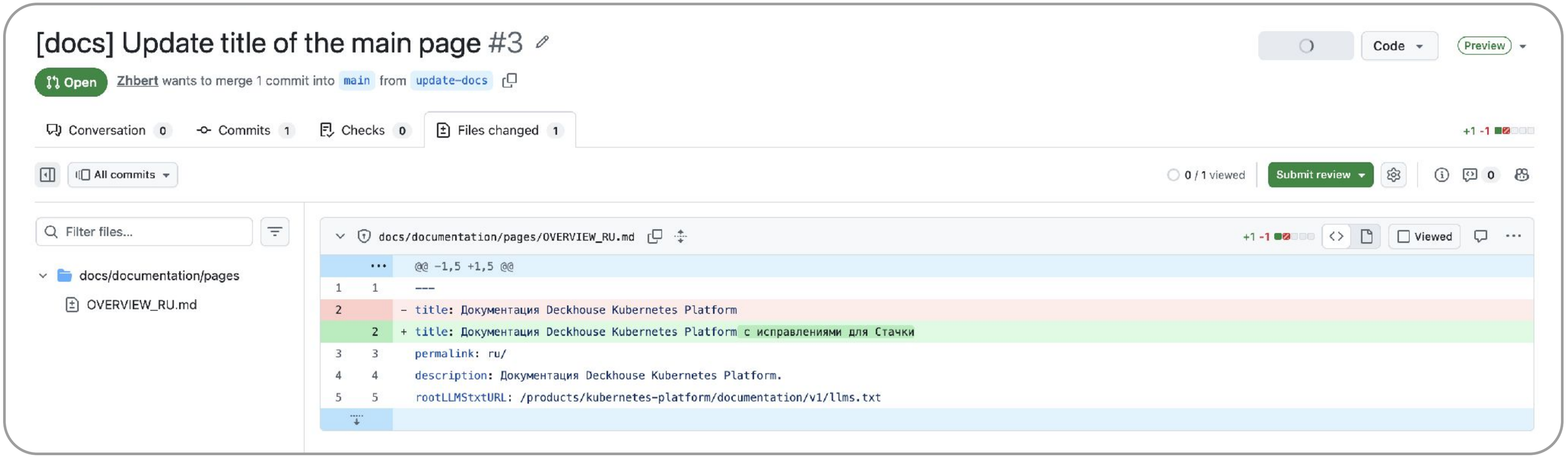
Оформляем запрос на слияние



08 Оформляем запрос на слияние



08 Оформляем запрос на слияние



08 Оформляем запрос на слияние



Выводы

Git — это не так уж и сложно!

- Даёт полную историю проекта
- Позволяет работать совместно, не мешая друг другу
- Можно легко починить сломанное

deckhouse СТАЧКА

3–4 октября ▪ Санкт-Петербург

Оцените доклад

@deckhouse_news

github.com/deckhouse

